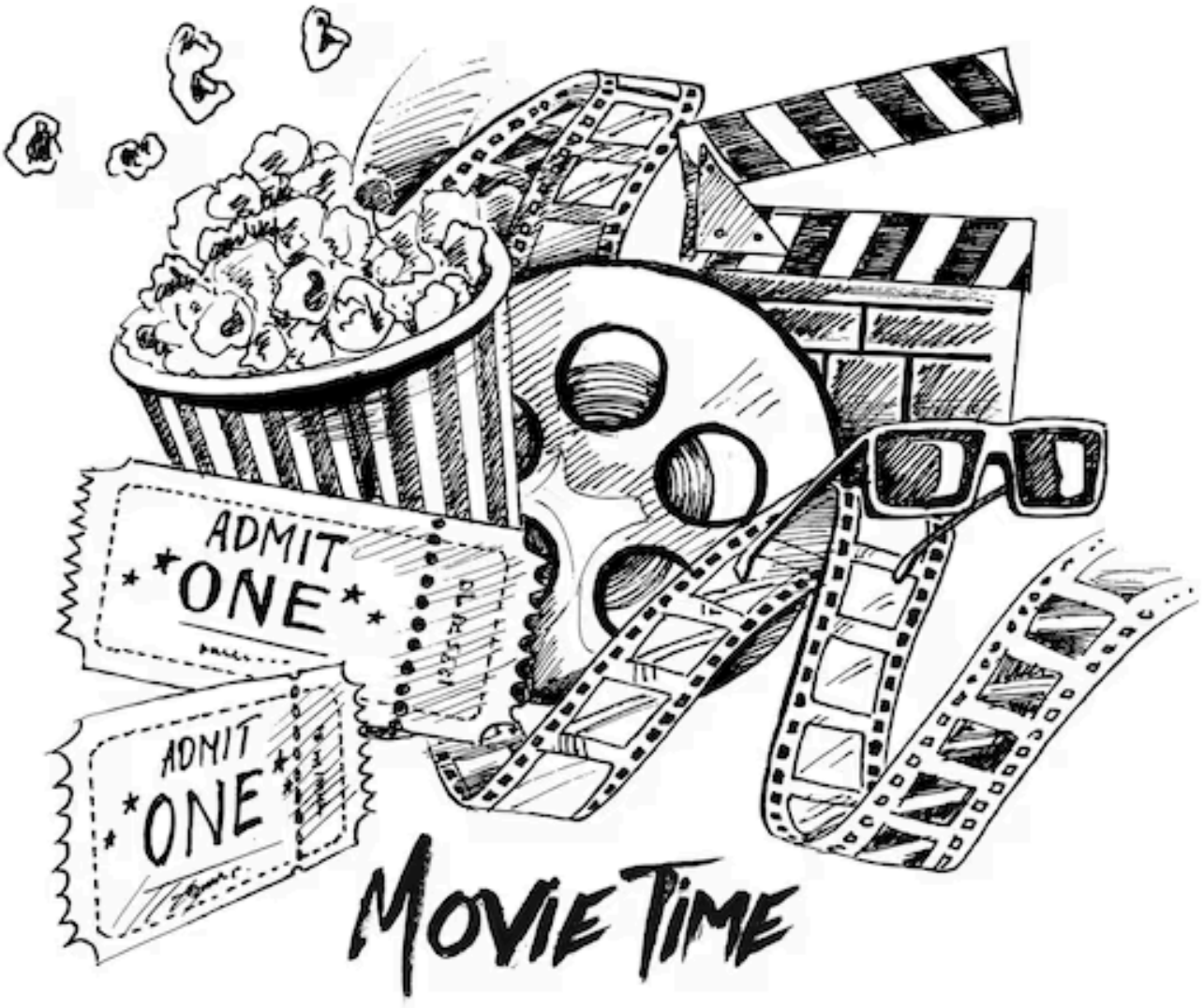
# Sujet SAÉ S3.01 − Application de Films

Groupe D : Hugo Ribeiro, Adam Benelcadi,
Ali Ouahabi,Achour Hamou Yahia



# LIVRABLE 3 :
# Mise en place d'une plateforme de développement

Groupe D:
- Hugo Ribeiro
- Adam Benelcadi
- Ali Ouahabi
- Achour Hamou Yahia

## 1. Présentation de la plateforme de développement :

La plateforme de développement de notre application web reposera sur HTML5, CSS3, Bootstrap, Bootstrap Icons, JavaScript et jQuery pour l'interface utilisateur. Côté serveur, ça sera PHP et Apache2. Pour le stockage des données, ça sera MariaDB (issues d'IMDb et éventuellement complétées par l'API TMDb). Comme tous les membres du groupe ne sont pas sur le même système d'exploitation, on utilisera Docker et Docker Compose pour faciliter la mise en place d'un environnement commun (en l'occurrence Windows et Linux). Visual Studio Code, Git et GitHub permettront d'organiser le développement collaboratif, tandis que de l'autre côté Composer va gérer les bibliothèques PHP et Adminer permet de gérer la base de données. Enfin, on pourra éventuellement envisager d'utiliser Xdebug pour le débogage, PHP_CodeSniffer pour la qualité du code, PHPUnit pour les tests automatisés, et enfin GitHub Actions pour l'intégration continue.

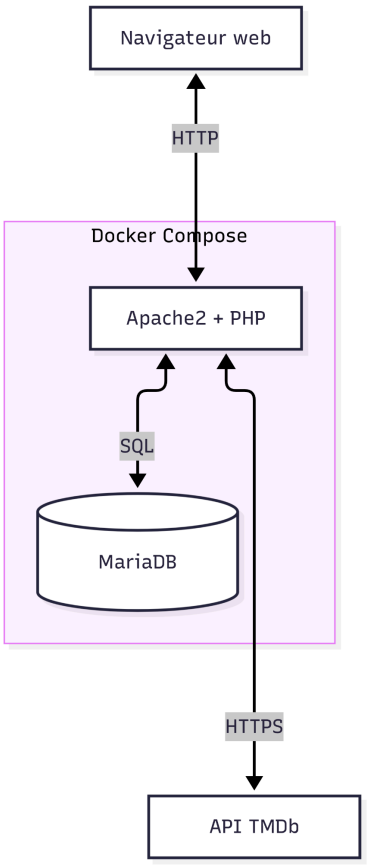
## 2. Architecture technique de l'application :

### 2.1. Organisation générale :

- **Frontend :** HTML5, CSS3, Bootstrap, Bootstrap Icons, JavaScript et jQuery.
- **Backend :** PHP exécuté par Apache2.
- **Base de données :** MariaDB, contenant les données cinématographiques importées depuis IMDb.

L'application pourra également communiquer avec l'API TMDb pour récupérer des informations complémentaires.

### 2.2. Schéma illustratif:




## 3. Installation et configuration de l'environnement de développement sur les postes des membres de l'équipe :

### 3.1. Installation des logiciels nécessaires :

| Logiciel | Étapes d'installation |
|---|---|
| Visual Studio Code | Télécharger l'installeur depuis code.visualstudio.com, l'exécuter et suivre les instructions. |
| Git | Télécharger l'installeur depuis git-scm.com, l'exécuter et suivre les instructions. |
| Docker Desktop | Télécharger l'installeur depuis docker.com, l'exécuter et suivre les instructions.<br>L'installation peut nécessiter WSL 2:<br>● Ouvrer PowerShell en tant qu'administrateur<br>● Taper « wsl --install » |

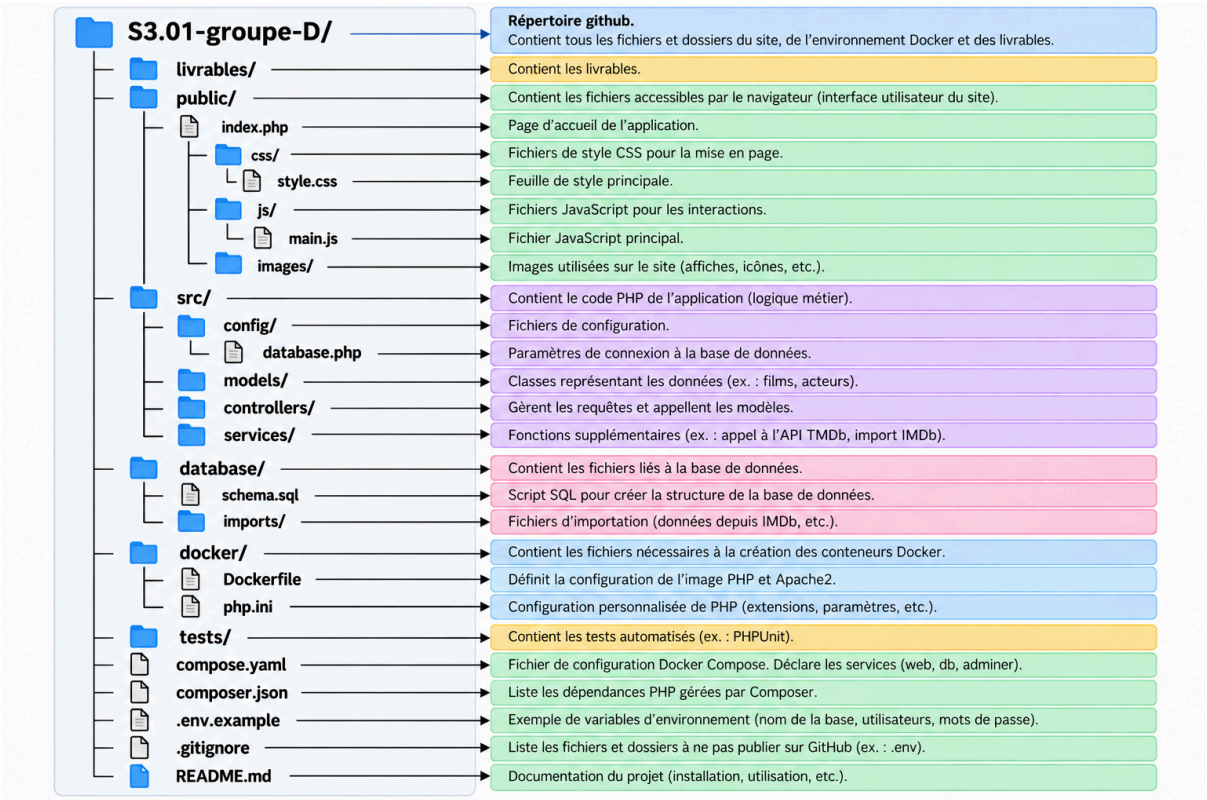
| | |
|---|---|
| Docker Compose | Inclus dans Docker Desktop sous Windows. Sous PowerShell vérifier son installation avec la commande « docker compose version ». |
| Visual Studio Code — extensions | Installer PHP Intelephense, PHP Debug et Docker depuis l'onglet Extensions. |

**/!\ Remarque:**

Docker Compose permet de regrouper les services nécessaires, à savoir : PHP, Apache2, MariaDB et Adminer, dans des conteneurs, sans avoir à les installer séparément sur chaque ordinateur des membres du groupe. Il pourra également intégrer Composer, Xdebug, PHP_CodeSniffer et PHPUnit. GitHub Actions, quant à lui, pourra être configuré directement sur GitHub pour automatiser les vérifications et les tests du projet.

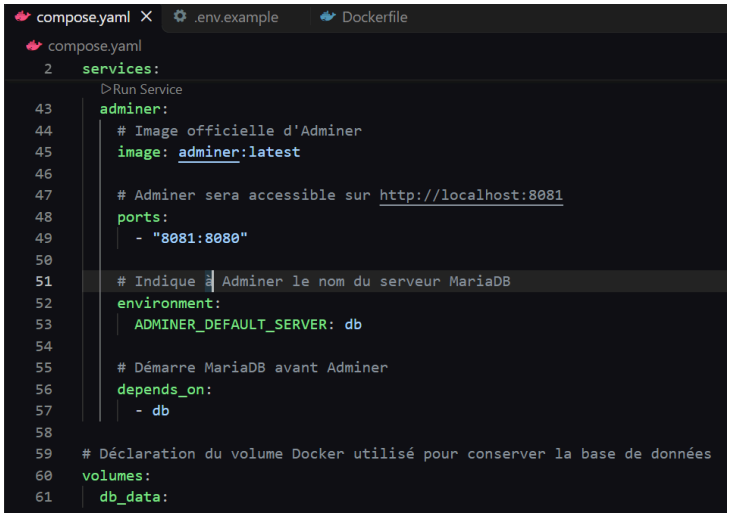
## 3.2. Configuration de l'environnement avec Docker Compose :

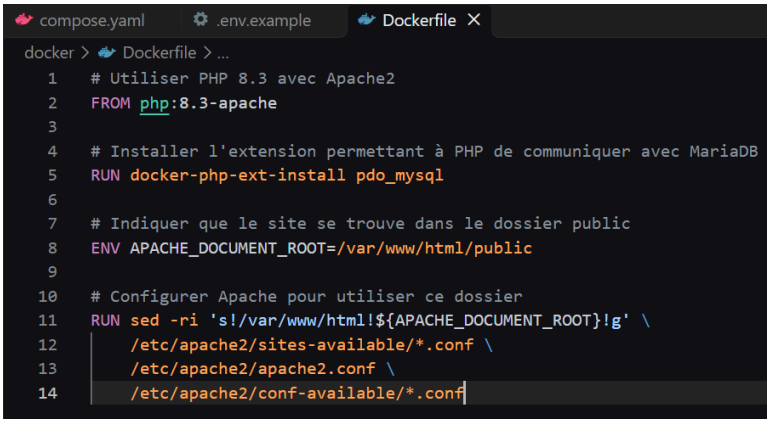
### 3.2.1. Préparation du répertoire :

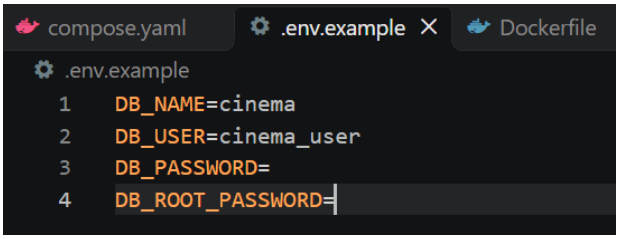


### 3.2.2. Configuration de Docker Compose :

On a donc configuré Docker Compose afin de disposer d'un environnement de développement commun à tous les membres du groupe (il va notamment regrouper PHP, Apache2, MariaDB et Adminer) dans des conteneurs. Le fichier **`compose.yaml`** permet de définir et de relier ces services. **Dockerfile** de son côté prépare l'environnement PHP et Apache2. Le fichier **`.env`** contient les paramètres de connexion à la base de données et reste PRIVE (!!!). Grâce à cette configuration, chaque membre du groupe peut alors récupérer le projet et démarrer les mêmes services avec la commande **`docker compose up -d --build`** , sans n'avoir à installer chaque logiciel séparément.




*Les captures suivantes présentent les fichiers utilisés pour configurer notre environnement Docker : **compose.yaml** pour les services, **Dockerfile** pour le serveur PHP/Apache2 et **.env.example** pour les paramètres de la base de données.*

### 3.3. Vérification du fonctionnement de l'environnement:

Pour vérifier notre configuration, on utilise la commande **`docker compose up -d --build`** afin de démarrer les conteneurs PHP/Apache2, MariaDB et Adminer. On vérifie ensuite leur état avec **`docker compose ps`** . On a également configuré deux fichiers pour les paramètres de connexion à la base de données. Le fichier **.env.example** sert de modèle partagé sur GitHub. Chaque membre du groupe doit copier son contenu dans un nouveau fichier **.env** sur son ordinateur, puis le remplir avec ses propres identifiants et mot de passe. Le fichier **.env** reste personnel et n'est pas envoyé sur GitHub grâce au fichier **.gitignore**.

Enfin, on teste l'accès à notre application sur http://localhost:8080 et à Adminer sur http://localhost:8081.

```
PS C:\Users\Pc\Desktop\S3.01-groupe-D> docker compose ps
NAME                     IMAGE              COMMAND                  SERVICE   CREATED          STATUS          PORTS
s301-groupe-d-adminer-1  adminer:latest     "entrypoint.sh docke…"   adminer   15 seconds ago   Up 14 seconds   0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
s301-groupe-d-db-1       mariadb:11.4       "docker-entrypoint.s…"   db        15 seconds ago   Up 14 seconds   3306/tcp
s301-groupe-d-web-1      s301-groupe-d-web  "docker-php-entrypoi…"   web       15 seconds ago   Up 14 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
PS C:\Users\Pc\Desktop\S3.01-groupe-D>
```

*Figure 1 : Vérification du démarrage des trois conteneurs avec la commande docker compose ps.*

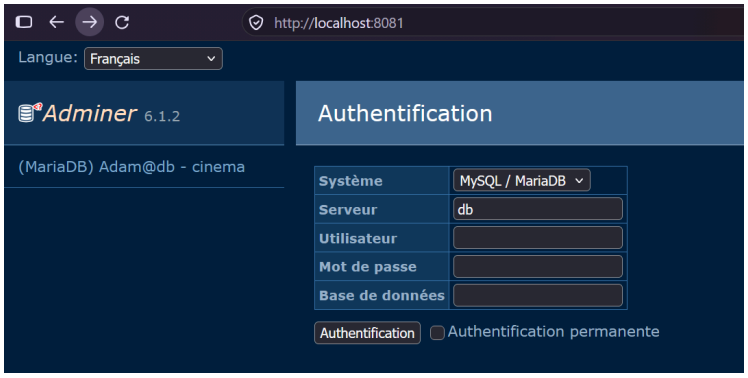



*Figure 2 : Accès à l'interface d'authentification d'Adminer sur le port 8081.*

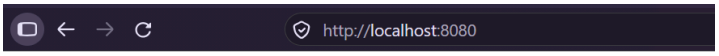



*Figure 3 : Accès à l'application web sur le port 8080 (page blanche = appli non encore développée).*

### 3.4. Configuration initiale du serveur :

Notre environnement de développement utilise les versions suivantes :

- Apache2 2.4.68 (Debian) : serveur web utilisé pour exécuter notre application.
- PHP 8.3.35 : langage utilisé pour le développement côté serveur.
- MariaDB 11.4.13 : système de gestion de base de données.
- Adminer 6.1.2 : interface permettant d'administrer la base de données.

On a vérifié les versions avec les commandes :

**`docker compose exec web php -v`**

**`docker compose exec web apache2 -v`** et

**`docker compose exec db mariadb --version`**

Le serveur Apache2 est configuré pour utiliser le dossier public/ comme répertoire racine de l'application. L'extension PHP pdo_mysql est installée pour permettre la communication avec MariaDB.

Comme nous avons choisi d'utiliser Docker pour notre environnement de développement, nous n'avons pas eu besoin d'installer et de configurer les modules Apache2 un par un. Ils sont déjà présents et activés dans notre conteneur Docker comme déjà mentionné. Nous avons vérifié leur présence avec la commande docker **`compose exec web apache2ctl -M`**. Parmi les modules compris, on retrouve par exemple :

- **php_module** : permet d'exécuter le code PHP.
- **mpm_prefork_module** : permet de gérer plusieurs requêtes simultanément.
- **dir_module** : permet de charger automatiquement un fichier comme index.php.
- **mime_module** : permet de reconnaître les différents types de fichiers (HTML, CSS, images).
- **auth_basic_module** : permet l'authentification HTTP.
- **authz_core_module** : gère les autorisations d'accès.
- **deflate_module** : permet de compresser les réponses HTTP.
- …etc